

Lecture 9

Fundamentals of Numerical Analysis

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

What we'll do:

- Gaussian elimination
- Conditioning

Effect of Perturbations to b

Consider the solution of a 2×2 system where

$$b = \begin{bmatrix} 1 \\ 2/3 \end{bmatrix}$$

One expects that the *exact* solutions to

$$Ax = \begin{bmatrix} 1 \\ 2/3 \end{bmatrix} \quad \text{and} \quad Ax = \begin{bmatrix} 1 \\ 0.6667 \end{bmatrix}$$

will be different. Should these solutions be a **lot different** or a **little different**?

Norms

Vectors:

$$\|x\|_p = (|x_1|^p + |x_2|^p + \dots + |x_n|^p)^{1/p}$$

$$\|x\|_1 = |x_1| + |x_2| + \dots + |x_n| = \sum_{i=1}^n |x_i|$$

$$\|x\|_\infty = \max(|x_1|, |x_2|, \dots, |x_n|) = \max_i (|x_i|)$$

Matrices:

$$\|A\| = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}$$

$$\|A\|_p = \max_{x \neq 0} \frac{\|Ax\|_p}{\|x\|_p}$$

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}|$$

$$\|A\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|$$

Effect of Perturbations to b

Perturb b with δb such that

$$\frac{\|\delta b\|}{\|b\|} \ll 1,$$

The perturbed system is

$$A(x + \delta x_b) = b + \delta b$$

The perturbations satisfy

$$A\delta x_b = \delta b$$

Analysis shows (see next two slides for proof) that

$$\frac{\|\delta x_b\|}{\|x\|} \leq \|A\| \|A^{-1}\| \frac{\|\delta b\|}{\|b\|}$$

Thus, the effect of the perturbation is small *only if* $\|A\| \|A^{-1}\|$ is small.

$$\frac{\|\delta x_b\|}{\|x\|} \ll 1 \quad \text{only if} \quad \|A\| \|A^{-1}\| \sim 1$$

Effect of Perturbations to b (Proof)

Let $x + \delta x_b$ be the *exact* solution to the perturbed system

$$A(x + \delta x_b) = b + \delta b \quad (1)$$

Expand

$$Ax + A\delta x_b = b + \delta b$$

Subtract Ax from left side and b from right side since $Ax = b$

$$A\delta x_b = \delta b$$

Left multiply by A^{-1}

$$\delta x_b = A^{-1}\delta b \quad (2)$$

Effect of Perturbations to b (Proof, p. 2)

Take norm of equation (2)

$$\|\delta x_b\| = \|A^{-1} \delta b\|$$

Applying consistency requirement of matrix norms

$$\|\delta x\| \leq \|A^{-1}\| \|\delta b\| \quad (3)$$

Similarly, $Ax = b$ gives $\|b\| = \|Ax\|$, and

$$\|b\| \leq \|A\| \|x\| \quad (4)$$

Rearrangement of equation (4) yields

$$\frac{1}{\|x\|} \leq \frac{\|A\|}{\|b\|} \quad (5)$$

Effect of Perturbations to b (Proof)

Multiply Equation (4) by Equation (3) to get

$$\frac{\|\delta x_b\|}{\|x\|} \leq \|A\| \|A^{-1}\| \frac{\|\delta b\|}{\|b\|} \quad (6)$$

Summary:

If $x + \delta x_b$ is the *exact* solution to the perturbed system

$$A(x + \delta x_b) = b + \delta b$$

then

$$\frac{\|\delta x_b\|}{\|x\|} \leq \|A\| \|A^{-1}\| \frac{\|\delta b\|}{\|b\|}$$

Effect of Perturbations to A

Perturb A with δA such that

$$\frac{\|\delta A\|}{\|A\|} \ll 1,$$

The perturbed system is

$$(A + \delta A)(x + \delta x_A) = b$$

Analysis shows that

$$\frac{\|\delta x_A\|}{\|x + \delta x_A\|} \leq \|A\| \|A^{-1}\| \frac{\|\delta A\|}{\|A\|}$$

Thus, the effect of the perturbation is small *only if* $\|A\| \|A^{-1}\|$ is small.

$$\frac{\|\delta x_A\|}{\|x + \delta x_A\|} \ll 1 \quad \text{only if} \quad \|A\| \|A^{-1}\| \sim 1$$

Effect of Perturbations to both A and b

Perturb both A with δA and b with δb such that

$$\frac{\|\delta A\|}{\|A\|} \ll 1 \quad \text{and} \quad \frac{\|\delta b\|}{\|b\|} \ll 1$$

The perturbation satisfies

$$(A + \delta A)(x + \delta x) = b + \delta b$$

Analysis shows that

$$\frac{\|\delta x\|}{\|x + \delta x\|} \leq \frac{\|A\| \|A^{-1}\|}{1 - \|A\| \|A^{-1}\| \frac{\|\delta A\|}{\|A\|}} \left[\frac{\|\delta A\|}{\|A\|} + \frac{\|\delta b\|}{\|b\|} \right]$$

Thus, the effect of the perturbation is small *only if* $\|A\| \|A^{-1}\|$ is small.

$$\frac{\|\delta x\|}{\|x + \delta x\|} \ll 1 \quad \text{only if} \quad \|A\| \|A^{-1}\| \sim 1$$

Condition number of A

The **condition number**

$$\kappa(A) \equiv \|A\| \|A^{-1}\|$$

indicates the sensitivity of the solution to perturbations in A and b . The condition number can be measured with any p -norm.

The condition number is always in the range

$$1 \leq \kappa(A) \leq \infty$$

- $\kappa(A)$ is a mathematical property of A
- Any algorithm will produce a solution that is sensitive to perturbations in A and b if $\kappa(A)$ is large.
- In exact math a matrix is either singular or non-singular.
 $\kappa(A) = \infty$ for a singular matrix
- $\kappa(A)$ indicates how close A is to being numerically singular.
- A matrix with large κ is said to be **ill-conditioned**

Computational Stability

In Practice, applying Gaussian elimination with partial pivoting and back substitution to $Ax = b$ gives the **exact solution**, $\hat{x}$, to the **nearby problem**

$$(A + E)\hat{x} = b \quad \text{where} \quad \|E\|_{\infty} \leq \varepsilon_m \|A\|_{\infty}$$

Gaussian elimination with partial pivoting and back substitution “gives exactly the right answer to nearly the right question.”

— Trefethen and Bau

Computational Stability

An algorithm that gives the exact answer to a problem that is near to the original problem is said to be **backward stable**. Algorithms that are not backward stable will tend to amplify roundoff errors present in the original data. As a result, the solution produced by an algorithm that is not backward stable will not necessarily be the solution to a problem that is close to the original problem.

Gaussian elimination without partial pivoting is *not* backward stable for arbitrary A .

If A is symmetric and positive definite, then Gaussian elimination without pivoting is backward stable.

The Residual

Let $\hat{x}$ be the numerical solution to $Ax = b$. $\hat{x} \neq x$ (x is the exact solution) because of roundoff.

The **residual** measures how close $\hat{x}$ is to satisfying the original equation

$$r = b - A\hat{x}$$

It is not hard to show that

$$\frac{\|\hat{x} - x\|}{\|\hat{x}\|} \leq \kappa(A) \frac{\|r\|}{\|b\|}$$

Small $\|r\|$ does not guarantee a small $\|\hat{x} - x\|$.

If $\kappa(A)$ is large the $\hat{x}$ returned by Gaussian elimination and back substitution (or any other solution method) is *not* guaranteed to be anywhere near the true solution to $Ax = b$.

Rules of Thumb

- Applying Gaussian elimination with partial pivoting and back substitution to $Ax = b$ yields a numerical solution $\hat{x}$ such that the residual vector $r = b - A\hat{x}$ is small *even if* the $\kappa(A)$ is large.
- If A and b are stored to machine precision ε_m , the numerical solution to $Ax = b$ by any variant of Gaussian elimination is correct to d digits where

$$d = |\log_{10}(\varepsilon_m)| - \log_{10}(\kappa(A))$$

Rules of Thumb

$$d = |\log_{10}(\varepsilon_m)| - \log_{10}(\kappa(A))$$

Example:

MATLAB computations have $\varepsilon_m \approx 2.2 \times 10^{-16}$. For a system with $\kappa(A) \sim 10^{10}$ the elements of the solution vector will have

$$\begin{aligned} d &= |\log_{10}(2.2 \times 10^{-16})| - \log_{10}(10^{10}) \\ &= 16 - 11 \\ &= 5 \end{aligned}$$

correct (decimal) digits

Summary of Limits to Numerical Solution of $Ax = b$

- 1 $\kappa(A)$ indicates how close A is to being numerically singular
- 2 If $\kappa(A)$ is “large”, A is **ill-conditioned** and *even the best* numerical algorithms will produce a solution, $\hat{x}$ that cannot be guaranteed to be close to the true solution, x
- 3 In practice, Gaussian elimination with partial pivoting and back substitution produces a solution with a small residual

$$r = b - A\hat{x}$$

even if $\kappa(A)$ is large.

The Backslash Operator

Consider the scalar equation

$$5x = 20 \quad \implies \quad x = (5)^{-1}20$$

The extension to a system of equations is, of course

$$Ax = b \quad \implies \quad x = A^{-1}b$$

where $A^{-1}b$ is the formal solution to $Ax = b$

In MATLAB notation the system is solved with

```
x = A\b
```

The Backslash Operator

Given an $n \times n$ matrix A , and an $n \times 1$ vector b the $\backslash$ operator performs a sequence of tests on the A matrix. MATLAB attempts to solve the system with the method that gives the least roundoff and the fewest operations.

When A is an $n \times n$ matrix:

- 1 MATLAB examines A to see if it is a permutation of a triangular system
If so, the appropriate triangular solve is used.
- 2 MATLAB examines A to see if it *appears* to be symmetric and positive definite.
If so, MATLAB attempts a Cholesky factorization and two triangular solves.
- 3 If the Cholesky factorization fails, or if A does not appear to be symmetric,
MATLAB attempts an LU factorization and two triangular solves.

More Algorithms for Special Systems

- tridiagonal systems
- banded systems
- LU decomposition
- Cholesky factorization

Tridiagonal

A tridiagonal matrix A

$$\begin{bmatrix} d_1 & c_1 & & & & & \\ a_1 & d_2 & c_2 & & & & \\ & a_2 & d_3 & c_3 & & & \\ & & \dots & \dots & \dots & & \\ & & & a_{i-1} & d_i & c_i & \\ & & & & \dots & \dots & \dots \\ & & & & \dots & \dots & \dots \\ & & & & & a_{n-1} & d_n \end{bmatrix}$$

- storage is saved by not saving zeros
- only $n + 2(n - 1) = 3n - 2$ places are needed to store the matrix versus n^2 for the whole system
- can operations be saved? yes!

Tridiagonal

$$\begin{bmatrix} d_1 & c_1 & & & & & \\ a_1 & d_2 & c_2 & & & & \\ & a_2 & d_3 & c_3 & & & \\ & & \dots & \dots & \dots & & \\ & & & a_{i-1} & d_i & c_i & \\ & & & & \dots & \dots & \dots \\ & & & & \dots & \dots & \dots \\ & & & & & a_{n-1} & d_n \end{bmatrix}$$

Start forward elimination (without any special pivoting)

- ➊ subtract a_1/d_1 times row 1 from row 2
- ➋ this eliminates a_1 , changes d_2 and does not touch c_2
- ➌ continuing:

$$d_i = d_i - \left(\frac{a_{i-1}}{d_{i-1}} c_{i-1} \right)$$

for $i = 2 \dots n$

Tridiagonal

$$\begin{bmatrix} \tilde{d}_1 & c_1 & & & & \\ & \tilde{d}_2 & c_2 & & & \\ & & \tilde{d}_3 & c_3 & & \\ & & & \dots & \dots & \\ & & & & \tilde{d}_i & c_i \\ & & & & & \dots & \dots \\ & & & & & & \dots & \dots \\ & & & & & & & \tilde{d}_n \end{bmatrix}$$

This leaves an upper triangular (2-band). With back substitution:

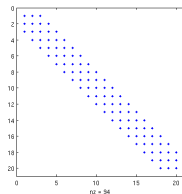
- ❶ $x_n = \tilde{b}_n / \tilde{d}_n$
- ❷ $x_{n-1} = (1/\tilde{d}_{n-1})(\tilde{b}_{n-1} - c_{n-1}x_n)$
- ❸ $x_i = (1/\tilde{d}_i)(\tilde{b}_i - c_i x_{i+1})$

Tridiagonal Algorithm

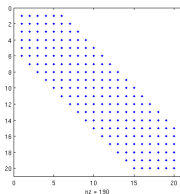
```
1 input: n, a, d, c, b
2 for i = 2 to n
3     xmult = ai-1 / di-1
4     di = di - xmult · ci-1
5     bi = bi - xmult · bi-1
6 end
7 xn = bn / dn
8 for i = n - 1 down to 1
9     xi = (bi - cixi+1) / di
10 end
```

Challenge: Will this algorithm make good use of the processor cores in a multicore processor? Why or why not?

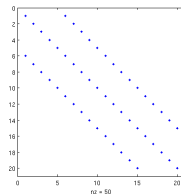
m -band



$$m = 5$$



$$m = 11$$

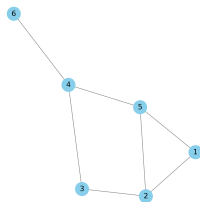


$$m = 11$$

- the m correspond to the total width of the non-zeros
- after a few passes of GE *fill-in* with occur within the band
- so an empty band costs (about) the same as a non-empty band
- one fix: reordering (e.g. Cuthill-McKee)
- generally GE will cost $\mathcal{O}(m^2n)$ for m -band systems

Motivation: Graph Theory

- Given a graph, construct associated matrix, called the graph Laplacian
- Row i of matrix corresponds to node i of graph
 - Diagonal entry is valence (total # of edges) for node i
 - Place a negative one in column j if node j is connected to i



$$\Rightarrow \begin{bmatrix} 2 & -1 & 0 & 0 & -1 & 0 \\ -1 & 3 & -1 & 0 & -1 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ -1 & -1 & 0 & -1 & 3 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{bmatrix}$$

Motivation: Graph Theory

Graph is Laplacian useful for

- Calculating spanning trees
- Partitioning a graph evenly
- and many more....

To use the graph Laplacian, you need to solve $Ax = b$ for many different vectors, b .

Motivation: Graph Theory (Multiple Right Hand Sides)

- Solve $Ax = b$ for many different b vectors
- For k different b vectors, Gaussian Elimination costs $\mathcal{O}(kn^3)$
- We can do better: LU factorization

Motivation: Symmetric Matrix

- A is symmetric, if $A = A^T$
- If $A = LU$ and A is symmetric, then could $L = U^T$?
- If so, this could save 50% of the computation of LU by only calculating L
- Save 50% of the FLOPS!
- This is achievable: LDL^T and Cholesky ($L^T L$) factorization

Factorization Methods

Factorizations are the common approach to solving $Ax = b$: simply organized Gaussian elimination.

What's next:

- LU factorization
- Cholesky factorization
- Use of the backslash operator

LU Factorization

Find L and U such that

$$A = LU$$

and L is lower triangular, and U is upper triangular.

$$L = \begin{bmatrix} 1 & 0 & \cdots & & 0 \\ \ell_{2,1} & 1 & 0 & & 0 \\ \ell_{3,1} & \ell_{3,2} & 1 & & 0 \\ \vdots & \vdots & & \ddots & \vdots \\ \ell_{n,1} & \ell_{n,2} & \cdots & \ell_{n-1,n} & 1 \end{bmatrix}$$

$$U = \begin{bmatrix} u_{1,1} & u_{1,2} & u_{1,3} & \cdots & u_{1,n} \\ 0 & u_{2,2} & u_{2,3} & \cdots & u_{2,n} \\ 0 & 0 & \ddots & \ddots & \vdots \\ \vdots & \vdots & & & u_{n-1,n} \\ 0 & 0 & & & u_{n,n} \end{bmatrix}$$

Since L and U are triangular, it is easy to apply their inverses.

Why?

- Since L and U are triangular, it is easy, $\mathcal{O}(n^2)$, to apply their inverses
- Decompose once, solve k right-hand sides quickly:
 - $\mathcal{O}(kn^3)$ with GE
 - $\mathcal{O}(n^3 + kn^2)$ with LU
- Given $A = LU$ you can compute A^{-1} , $\det(A)$, $\text{rank}(A)$, $\ker(A)$, etc...

LU Factorization

Since L and U are triangular, it is easy to apply their inverses.
Consider the solution to $Ax = b$.

$$A = LU \implies (LU)x = b$$

Regroup since matrix multiplication is associative

$$L(Ux) = b$$

Let $Ux = y$, then

$$Ly = b$$

Since L is triangular it is easy (without Gaussian elimination) to compute

$$y = L^{-1}b$$

This expression should be interpreted as “Solve $Ly = b$ with forward substitution.”

LU Factorization

Now, since y is known, solve for x

$$x = U^{-1}y$$

which is interpreted as “Solve $Ux = y$ with backward substitution.”

LU Factorization

Listing 1: LU Solve

```
1 Factor  $A$  into  $L$  and  $U$ 
2 Solve  $Ly = b$  for  $y$            use forward substitution
3 Solve  $Ux = y$  for  $x$            use backward substitution
```

LU Factorization

- If we have $Ax = b$ and perform GE we end up with

$$A = \begin{bmatrix} x & x & x & x \\ x & x & x & x \\ x & x & x & x \\ x & x & x & x \end{bmatrix} \Rightarrow \begin{bmatrix} x' & x' & x' & x' \\ 0 & x' & x' & x' \\ 0 & 0 & x' & x' \\ 0 & 0 & 0 & x' \end{bmatrix}$$

- Remember from Lectures on Sept 8 and 10, that naive Gaussian Elimination can be done by matrix multiplication

$$MAx = Mb$$

$$Ux = Mb$$

- MA is upper triangular and called U
- M is the elimination matrix

LU Factorization

As an example take one column step of GE, A becomes

$$\begin{bmatrix} 6 & -2 & 2 & 4 \\ 12 & -8 & 6 & 10 \\ 3 & -13 & 9 & 3 \\ -6 & 4 & 1 & -18 \end{bmatrix} \rightarrow \begin{bmatrix} 6 & -2 & 2 & 4 \\ 0 & -4 & 2 & 2 \\ 0 & -12 & 8 & 1 \\ 0 & 2 & 3 & -14 \end{bmatrix}$$

using the elimination matrix

$$M_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ -\frac{1}{2} & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}$$

So we have performed

$$M_1 A x = M_1 b$$

LU Factorization

From Lectures on Sept 8 and 10

- Inverting M_i is easy: just flip the sign of the lower triangular entries

$$M_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ -\frac{1}{2} & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix} \Rightarrow M_1^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ \frac{1}{2} & 0 & 1 & 0 \\ -1 & 0 & 0 & 1 \end{bmatrix}$$

- M_i^{-1} is just the multipliers used in Gaussian Elimination!
- $M_i^{-1}M_j^{-1}$ is still lower triangular, for $i < j$,
and is the union of the columns
- $M_1^{-1}M_2^{-1} \dots M_j^{-1}$ is lower triangular, with the lower triangle the multipliers from Gaussian Elimination

LU Factorization

- Zeroing each column yields another elimination matrix operation:

$$M_3M_2M_1Ax = M_3M_2M_1b$$

- $M = M_3M_2M_1$. Thus
- $L = M_1^{-1}M_2^{-1}M_3^{-1}$ is lower triangular

$$MA = U$$

$$M_3M_2M_1A = U$$

$$A = M_1^{-1}M_2^{-1}M_3^{-1}U$$

$$A = LU$$

LU (forward elimination) Algorithm

Listing 2: LU

```
1  given  $A$ 
2
3  for  $k = 1 \dots n - 1$ 
4      for  $i = k + 1 \dots n$ 
5           $xmult = a_{ik}/a_{kk}$ 
6           $a_{ik} = xmult$ 
7          for  $j = k + 1 \dots n$ 
8               $a_{ij} = a_{ij} - (xmult)a_{kj}$ 
9          end
10     end
11 end
```

- U is stored in the upper triangular portion of A
- L (without the diagonal) is stored in the lower triangular

Doolittle Factorization (LU)

Listing 3: Doolittle

```
1  given  $A$ 
2  output  $L, U$ 
3
4  for  $k = 1 \dots n$ 
5       $\ell_{kk} = 1$ 
6      for  $j = k \dots n$ 
7           $u_{kj} = a_{kj} - \sum_{i=1}^{k-1} \ell_{ki} u_{ij}$ 
8      end
9      for  $j = k + 1 \dots n$ 
10          $\ell_{jk} = (a_{jk} - \sum_{i=1}^{k-1} \ell_{ji} u_{ik}) / u_{kk}$ 
11     end
12 end
```

- Mathematically the same as previous LU
- Difference is we now explicitly form L and U

What About Pivoting?

- Pivoting (that is row exchanges) can be expressed in terms of matrix multiplication
- Do pivoting during elimination, but track row exchanges in order to express pivoting with matrix P
- Let P be all zeros
 - Place a 1 in column j of row 1 to exchange row 1 and row j
 - If no row exchanged needed, place a 1 in column 1 of row 1
 - *Repeat for all rows of P*
- P is a permutation matrix
- Now using pivoting,

$$LU = PA$$

MATLAB *LU*

Like GE, *LU* needs pivoting. With pivoting the *LU* factorization always exists, even if *A* is singular. With pivoting, we get

$$LU = PA$$

```
1 >> A=rand(4,4);
2 >> b=rand(4,1);
3 >> [L,U,P]=lu(A)
4 L = 1.0000    0    0    0
5     0.9013    1.0000    0    0
6     0.0298   -0.8982    1.0000    0
7     0.7233    0.5813   -0.2670    1.0000
8 U = 0.7809    0.9890    0.4613    0.2971
9     0   -0.8838   -0.0548    0.1857
10    0    0    0.7183    0.6403
11    0    0    0    0.2065
12 P = 0    1    0    0
13     1    0    0    0
14     0    0    0    1
15     0    0    1    0
16 >> x=U\(L\(P*b))
17 x = 0.5326 0.5416 -1.2765 1.1315
18 >> A\b
19     0.5326 0.5416 -1.2765 1.1315
```